ELASTICSEARCH WORKSHOP

Category 3: Keyword and Term-Level Queries

EXERCISES

Scenario

You are a data engineer for a fictional intelligence unit called UNIT-7.
Your unit maintains a watch-list of persons of interest.
You will search this list using exact values, code prefixes, wildcards, and field checks.
All names, codes, and locations are completely fictional.

Key Concept: Term-Level Queries

Term-level queries search for exact values. They do not analyze the text.
They work on keyword, integer, boolean, and date fields.
They do NOT work well on text fields. Use match queries for text fields.

Query type	What it does
`term`	Match one exact value. Like SQL WHERE col = 'value'.
`terms`	Match any value from a list. Like SQL WHERE col IN ('a','b','c').
`exists`	Match documents where a field has any value. Like SQL WHERE col IS NOT NULL.
`prefix`	Match keyword fields that start with a given string.
`wildcard`	Match keyword fields using * (any characters) or ? (one character).
`ids`	Match documents by their document ID.

Important rule: term queries are case-sensitive.
Searching for 'colonel' will NOT find 'Colonel'.
Always use the exact value as it was stored.

SQL analogy: term = WHERE col = 'value' (exact match, case-sensitive)
MongoDB analogy: term = db.col.find({ field: 'value' })

Dataset: Persons of Interest Watch-List

Index name: 'watch_list'

You will index 10 fictional persons of interest. Fields:

Field	Type	Notes
subject_id	keyword	Code like 'POI-001'. Used as document ID.
alias	keyword	Known alias. Exact match only.
nationality	keyword	Country. Exact match only.
threat_level	keyword	One of: HIGH, MEDIUM, LOW.
unit_code	keyword	Internal unit code like 'UNIT-7A'.
last_seen	date	Date last seen. Format: YYYY-MM-DD.
armed	boolean	True if confirmed armed.
known_associates	keyword	List of subject_ids. Can be empty or missing.

The 10 records to index:

subject_id	alias	nationality	threat_level	unit_code	last_seen	armed
POI-001	Ghost	Syrian	HIGH	UNIT-7A	2024-02-10	true
POI-002	Falcon	Lebanese	MEDIUM	UNIT-7B	2024-01-22	false
POI-003	Viper	Iranian	HIGH	UNIT-7A	2024-03-05	true
POI-004	Shadow	Syrian	LOW	UNIT-7C	2023-12-18	false
POI-005	Hawk	Turkish	MEDIUM	UNIT-7B	2024-02-28	true
POI-006	Dagger	Iranian	HIGH	UNIT-7A	2024-03-11	true
POI-007	Cobra	Lebanese	LOW	UNIT-7C	2023-11-30	false
POI-008	Phantom	Turkish	MEDIUM	UNIT-7B	2024-01-15	false
POI-009	Raven	Syrian	HIGH	UNIT-7A	2024-03-20	true
POI-010	Scorpion	Iranian	MEDIUM	UNIT-7C	2024-02-05	true

known_associates field: POI-001 knows POI-003 and POI-006. POI-003 knows POI-006. All others have no known_associates field (it is missing entirely).

Exercise 1 [Step-by-step]

Task: Create the 'watch_list' index with the correct mapping. Bulk index all 10 records.

Step 1 — Create the index

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

if es.indices.exists(index='watch_list'):
    es.indices.delete(index='watch_list')

mapping = {
    'mappings': {
        'properties': {
            'subject_id':        { 'type': 'keyword' },
            'alias':             { 'type': 'keyword' },
            'nationality':       { 'type': 'keyword' },
            'threat_level':      { 'type': 'keyword' },
            'unit_code':         { 'type': 'keyword' },
            'last_seen':         { 'type': 'date'    },
            'armed':             { 'type': 'boolean' },
            'known_associates':  { 'type': 'keyword' }
        }
    }
}

es.indices.create(index='watch_list', body=mapping)
print('Index created.')
```

Step 2 — Bulk index all 10 records

```
from elasticsearch.helpers import bulk

subjects = [
    { '_index': 'watch_list', '_id': 'POI-001', '_source': {
        'subject_id': 'POI-001', 'alias': 'Ghost',    'nationality': 'Syrian',
        'threat_level': 'HIGH',   'unit_code': 'UNIT-7A', 'last_seen':
'2024-02-10',
        'armed': True,  'known_associates': ['POI-003', 'POI-006'] } },
    { '_index': 'watch_list', '_id': 'POI-002', '_source': {
        'subject_id': 'POI-002', 'alias': 'Falcon',   'nationality': 'Lebanese',
        'threat_level': 'MEDIUM', 'unit_code': 'UNIT-7B', 'last_seen':
'2024-01-22',
        'armed': False } },
    { '_index': 'watch_list', '_id': 'POI-003', '_source': {
        'subject_id': 'POI-003', 'alias': 'Viper',    'nationality': 'Iranian',
        'threat_level': 'HIGH',   'unit_code': 'UNIT-7A', 'last_seen':
'2024-03-05',
        'armed': True,  'known_associates': ['POI-006'] } },
    { '_index': 'watch_list', '_id': 'POI-004', '_source': {
        'subject_id': 'POI-004', 'alias': 'Shadow',   'nationality': 'Syrian',
        'threat_level': 'LOW',    'unit_code': 'UNIT-7C', 'last_seen':
'2023-12-18',
        'armed': False } },
    { '_index': 'watch_list', '_id': 'POI-005', '_source': {
        'subject_id': 'POI-005', 'alias': 'Hawk',     'nationality': 'Turkish',
        'threat_level': 'MEDIUM', 'unit_code': 'UNIT-7B', 'last_seen':
'2024-02-28',
        'armed': True  } },
    { '_index': 'watch_list', '_id': 'POI-006', '_source': {
```

```
        'subject_id': 'POI-006', 'alias': 'Dagger',   'nationality': 'Iranian',
        'threat_level': 'HIGH',   'unit_code': 'UNIT-7A', 'last_seen':
'2024-03-11',
        'armed': True  } },
    { '_index': 'watch_list', '_id': 'POI-007', '_source': {
        'subject_id': 'POI-007', 'alias': 'Cobra',    'nationality': 'Lebanese',
        'threat_level': 'LOW',    'unit_code': 'UNIT-7C', 'last_seen':
'2023-11-30',
        'armed': False } },
    { '_index': 'watch_list', '_id': 'POI-008', '_source': {
        'subject_id': 'POI-008', 'alias': 'Phantom',  'nationality': 'Turkish',
        'threat_level': 'MEDIUM', 'unit_code': 'UNIT-7B', 'last_seen':
'2024-01-15',
        'armed': False } },
    { '_index': 'watch_list', '_id': 'POI-009', '_source': {
        'subject_id': 'POI-009', 'alias': 'Raven',    'nationality': 'Syrian',
        'threat_level': 'HIGH',   'unit_code': 'UNIT-7A', 'last_seen':
'2024-03-20',
        'armed': True  } },
    { '_index': 'watch_list', '_id': 'POI-010', '_source': {
        'subject_id': 'POI-010', 'alias': 'Scorpion', 'nationality': 'Iranian',
        'threat_level': 'MEDIUM', 'unit_code': 'UNIT-7C', 'last_seen':
'2024-02-05',
        'armed': True  } },
]

success, errors = bulk(es, subjects)
print('Indexed:', success, '| Errors:', errors)
# Expected: Indexed: 10 | Errors: []
```

Note: POI-002, POI-004, POI-005, POI-007, POI-008 have no known_associates field.
The field is simply not included in their document.
This is valid in Elasticsearch. Missing fields are not errors.
We will query for missing fields in Exercise 6.

Exercise 2 [Step-by-step] (required)

Task: Use a term query to find all subjects with threat_level 'HIGH'.

Print the subject_id and alias of each result.

Step 1 — Write the term query

A term query matches documents where a field has an exact value.
The value must match exactly, including case.

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')
```

```
# 'term' searches for an exact value in a keyword field.
# 'threat_level': 'HIGH' means: find documents where threat_level is exactly
'HIGH'.
# This will NOT match 'high' or 'High'. It is case-sensitive.
query = {
    'query': {
        'term': {
            'threat_level': 'HIGH'
        }
    }
}
```

Step 2 — Run the query and print results

```
response = es.search(index='watch_list', body=query)
hits = response['hits']['hits']

print('HIGH threat subjects:', len(hits))
for hit in hits:
    src = hit['_source']
    print(src['subject_id'], '-', src['alias'])

# Expected output:
# HIGH threat subjects: 4
# POI-001 - Ghost
# POI-003 - Viper
# POI-006 - Dagger
# POI-009 - Raven
```

```
Command reference:
  'term': { 'field_name': 'exact_value' }

  term queries are NEVER used on text fields.
  text fields are analyzed (split into words). The stored tokens are lowercase.
  Searching for 'Ghost' on a text field would fail because 'Ghost' gets stored as 'ghost'.
  Always use term on keyword, integer, boolean, or date fields.

SQL analogy:     SELECT * FROM watch_list WHERE threat_level = 'HIGH'
MongoDB analogy: db.watch_list.find({ threat_level: 'HIGH' })
```

Exercise 3 (required)

Task: Find all subjects whose nationality is either 'Iranian' or 'Lebanese'.

Use a terms query. terms lets you provide a list of values.
A document matches if its field value is any value in the list.

Print the subject_id, alias, and nationality of each result.

Expected result: POI-002, POI-003, POI-006, POI-007, POI-010 (5 subjects).

```
Command reference:
  'terms': { 'field_name': ['value1', 'value2', 'value3'] }

  terms accepts a list. A document matches if its value is IN the list.

SQL analogy:     WHERE nationality IN ('Iranian', 'Lebanese')
MongoDB analogy: db.watch_list.find({ nationality: { $in: ['Iranian','Lebanese'] } })
```

Exercise 4

Task: Find all subjects who are confirmed armed (armed = true).

Use a term query on the 'armed' boolean field.
Print the subject_id, alias, and threat_level of each result.

Expected result: POI-001, POI-003, POI-005, POI-006, POI-009, POI-010 (6 subjects).

```
Command reference:
  'term': { 'armed': True }

  Boolean fields use Python True/False (not the string 'true').
  In the JSON query body, use true (lowercase, no quotes).
  The Python client converts Python True to JSON true automatically.

SQL analogy:     WHERE armed = true
```

Exercise 5

Task: Retrieve the documents for POI-004, POI-007, and POI-009 in one query.

Use the ids query. It fetches documents by their document ID.
Print the subject_id, alias, and nationality of each result.

Expected result: Shadow (Syrian), Cobra (Lebanese), Raven (Syrian).

```
Command reference:
  'ids': { 'values': ['id1', 'id2', 'id3'] }

  The ids query matches documents by the _id field (the document ID).
  This is faster than a term query when you already know the exact IDs.

SQL analogy:     WHERE id IN ('POI-004', 'POI-007', 'POI-009')
MongoDB analogy: db.watch_list.find({ _id: { $in: ['POI-004','POI-007','POI-009'] } })
```

Exercise 6 (required)

Task: Find all subjects who HAVE a known_associates field.

Remember: most subjects do not have this field. It is missing from their document.
Use the exists query to find only the subjects where this field is present.

Print the subject_id, alias, and the known_associates list.

Expected result: POI-001 (Ghost) and POI-003 (Viper).

```
Command reference:
  'exists': { 'field': 'field_name' }

  exists returns documents where the named field exists and has a value.
  It does NOT return documents where the field is missing entirely.

SQL analogy:     WHERE known_associates IS NOT NULL
MongoDB analogy: db.watch_list.find({ known_associates: { $exists: true } })
```

Exercise 7

Task: Find all subjects whose unit_code starts with 'UNIT-7A'.

Use a prefix query on the unit_code field.
Print the subject_id, alias, and unit_code of each result.

Expected result: POI-001, POI-003, POI-006, POI-009 (all assigned to UNIT-7A).

```
Command reference:
  'prefix': { 'field_name': 'starting_value' }

  prefix matches keyword fields that START with the given string.
  Case-sensitive. 'UNIT-7' matches 'UNIT-7A', 'UNIT-7B', 'UNIT-7C'.
  'unit-7' (lowercase) would match nothing.

SQL analogy:     WHERE unit_code LIKE 'UNIT-7A%'
MongoDB analogy: db.watch_list.find({ unit_code: /^UNIT-7A/ })
```

Exercise 8

Task: Find all subjects whose alias ends with the letter 'r'.

Use a wildcard query on the alias field.
The * character means: any number of any characters.
So '*r' means: any characters, then the letter r at the end.

Print the subject_id and alias of each result.

Expected result: POI-003 (Viper), POI-010 (Scorpion does NOT end in r)... check the data carefully.

```
Command reference:
  'wildcard': { 'field_name': 'pattern' }

  * means zero or more of any character.
  ? means exactly one of any character.

  Example patterns:
    '*er'    matches any value that ends with 'er'
    'P*'     matches any value that starts with 'P'
    'P?I-*'  matches 'POI-001', 'PAI-002', etc.

  Warning: wildcard queries are slow on large indexes.
  Avoid leading wildcards like '*word' when possible.

SQL analogy:     WHERE alias LIKE '%r'
MongoDB analogy: db.watch_list.find({ alias: /r$/ })
```

Exercise 9

Task: Find the subject with alias 'Phantm' using a fuzzy query.

A fuzzy query finds values that are close to the search term, even if spelled differently.
'Phantm' is a typo of 'Phantom'. A fuzzy query will still find it.

Use a fuzzy query on the alias field with the value 'Phantm'.
Print the subject_id and alias of the result.

Expected result: POI-008 (Phantom).

```
Command reference:
  'fuzzy': { 'field_name': { 'value': 'typo_word', 'fuzziness': 'AUTO' } }

  fuzziness 'AUTO' means Elasticsearch decides how many character differences to allow.
  For words of 5+ characters, it allows up to 2 different characters.

  This is useful for search boxes where users may make typos.

SQL analogy: No direct SQL equivalent. SQL LIKE cannot do fuzzy matching.
```

Exercise 10

Task: Find all HIGH threat subjects. Return only 2 results. Show only the alias and nationality fields.

Use a term query for threat_level = 'HIGH'.
Add 'size' to the query to limit results to 2.
Add '_source' to the query to return only alias and nationality.

Expected result: 2 records, each showing only alias and nationality.

```
Command reference:
  Add to the query body:
    'size': 2                          Return at most 2 results.
    '_source': ['alias', 'nationality']  Return only these fields.

  size default is 10. Always set it explicitly in production.
```

```
_source filtering reduces network traffic. Only ask for what you need.

SQL analogy:     SELECT alias, nationality FROM watch_list
                 WHERE threat_level = 'HIGH' LIMIT 2
```

End of Category 3 Exercises